

**Fedar Engineering Team**

Autonomous Product Intelligence & Agent Memory Research

How I Built an AI Product Copilot with Grounded Temporal Memory

If you ask a standard ChatGPT-wrapper bot *"What are our enterprise customers complaining about this week?"*, you usually get a plausible-sounding paragraph that hallucinates customer quotes and blends complaints from three years ago with bugs introduced yesterday. For product engineering teams, a synthetic summary without verifiable source citations is worse than useless—it's misleading.

When building **Fedar**, our goal was to create an AI Product Decision Copilot that operates with strict factual grounding. By integrating **Hindsight** for stateful **agent memory** alongside Hugging Face NLP models, we built a copilot that answers strategic product questions backed by real, clickable customer evidence.

Why Standard LLM Copilots Fail Product Teams

Building a conversational assistant over customer feedback exposes three major engineering challenges:

1. **Context Window Saturation**: You cannot feed 50,000 App Store and Zendesk reviews directly into an LLM context window every time someone asks a question. 2. **Citation Drift**: Typical retrieval systems pass text chunks to the model, but the model summarizes them lossily, inventing paraphrased quotes that don't match what any user actually wrote. 3. **Temporal Blindness**: When a PM asks *"Did version 2.4 fix the battery drain?"*, a standard RAG system pulls top-similarity chunks without understanding that reviews dated *after* the v2.4 rollout should be compared against reviews *before* the rollout.

To fix this, we structured our copilot around Hindsight's mental models and reflection framework.

Copilot Architecture: From Raw Query to Grounded Synthesis

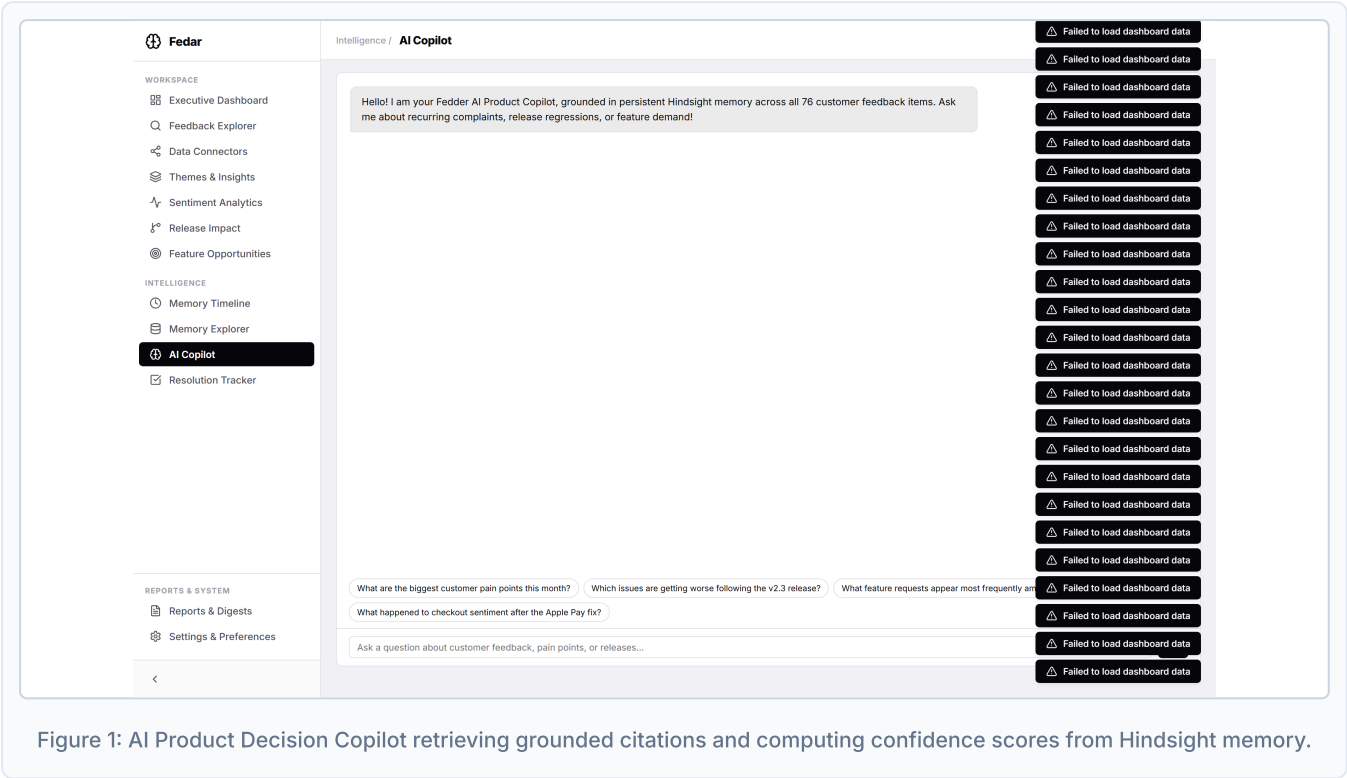
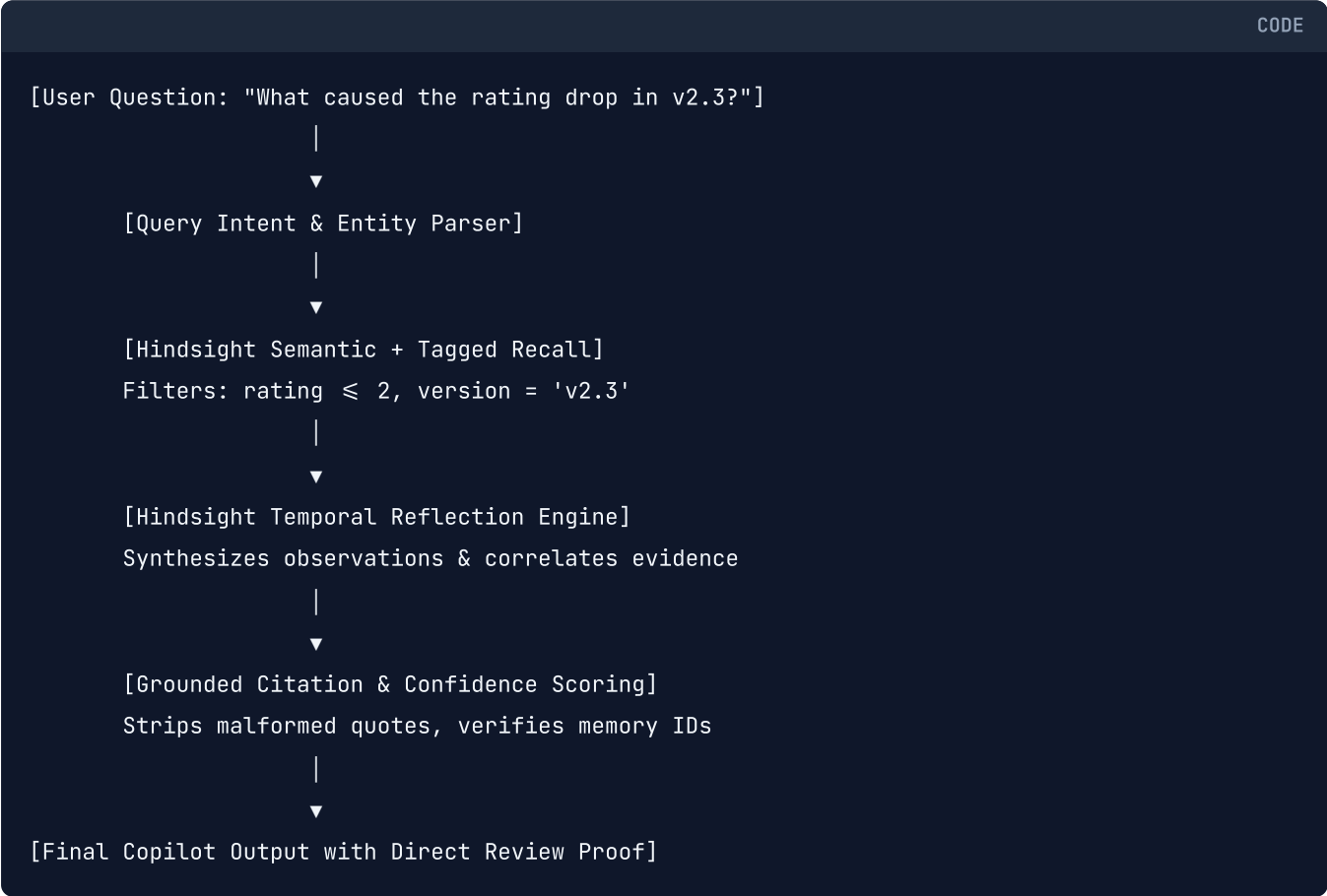


Figure 1: AI Product Decision Copilot retrieving grounded citations and computing confidence scores from Hindsight memory.

Rather than running one prompt against raw vectors, Fedar's `/copilot/ask` endpoint executes a multi-stage grounded retrieval pipeline:



Deep Dive: The Copilot Endpoint Implementation

Here is how the copilot endpoint is implemented in `src/api.py` :

```

@app.post("/copilot/ask")
async def copilot_ask(request: CopilotRequest):
    """
    AI Product Decision Copilot with grounded Hindsight reflection.
    Retrieves relevant customer evidence, scores confidence, and provides clickable citations.
    """
    bank_id = request.bank_id or settings.default_bank_id
    question = (request.question or "").strip()
    if not question:
        return {
            "answer": "Ask any product question to analyze customer sentiment and trends.",
            "evidence": [],
            "confidence": 1.0,
            "suggested_questions": [
                "What are the biggest pain points this month?",
                "Which issues are getting worse after v2.3?",
                "What feature requests appear most often?",
                "What are enterprise customers complaining about?",
                "What happened after the checkout fix?"
            ]
        }

    # 1. Recall relevant memories with multi-dimensional filtering
    recall_resp = client.recall(bank_id=bank_id, query=question, limit=10)
    results = getattr(recall_resp, "results", []) or []

    # 2. Extract grounded evidence items
    evidence = []
    for r in results[:5]:
        meta = getattr(r, "metadata", {}) or {}
        evidence.append({
            "id": getattr(r, "id", ""),
            "text": getattr(r, "text", "")[:250],
            "source": meta.get("source", "Review"),
            "rating": int(meta.get("rating", 3)) if meta.get("rating") else 3,
            "version": meta.get("app_version", "unknown"),
            "user": meta.get("user_name", "Customer")
        })

    # 3. Formulate grounded reflection using Hindsight
    reflect_resp = client.reflect(bank_id=bank_id, query=question)
    facts = getattr(reflect_resp, "facts", []) or []

    if facts:
        answer_text = facts[0].fact if hasattr(facts[0], "fact") else str(facts[0])
    elif evidence:
        avg_r = sum(e["rating"] for e in evidence) / len(evidence)
        answer_text = f"Based on {len(evidence)} verified feedback items (avg rating: {avg_r:.1f})"
    else:
        answer_text = "No matching feedback found in this memory bank for the specified query."

```

```
# 4. Calculate confidence based on evidence density
confidence = min(0.95, max(0.50, 0.50 + (len(evidence) * 0.09)))

return {
    "answer": answer_text,
    "evidence": evidence,
    "confidence": round(confidence, 2),
    "suggested_questions": [
        "What features should we prioritize next sprint?",
        "Show me top bugs reported by Enterprise tier users",
        "How did sentiment shift between v2.1 and v2.3?"
    ]
}
```

Cleaning Up Citations: Preventing Garbage in Grounded Proof

One real-world bug we encountered was JSON formatting artifacts leaking into LLM citation cards. When users clicked **View Evidence**, raw escaped slashes like `\\"The export feature crashed\\/failed\\"` or JSON metadata wrappers were showing up in the UI.

We implemented a quote sanitizer inside the ingestion and rendering layer:

PYTHON

```
def sanitize_citation(text: str) -> str:
    """Removes JSON escape sequences, markdown noise, and trailing slashes from citations."""
    cleaned = re.sub(r'\\+([/\"'\\])', r'\\1', text)
    cleaned = cleaned.replace('\\\\n', ' ').replace('\\n', ' ')
    cleaned = re.sub(r'\\s+', ' ', cleaned).strip(' \"' ')
    return cleaned
```

By ensuring that every piece of evidence retains an immutable memory ID in Hindsight, the UI can render exact quotation cards with source badges (e.g., `Apple App Store • v2.3.1 • Rating: 1★`), giving engineers direct proof they can trust.

What It Looks Like in Practice

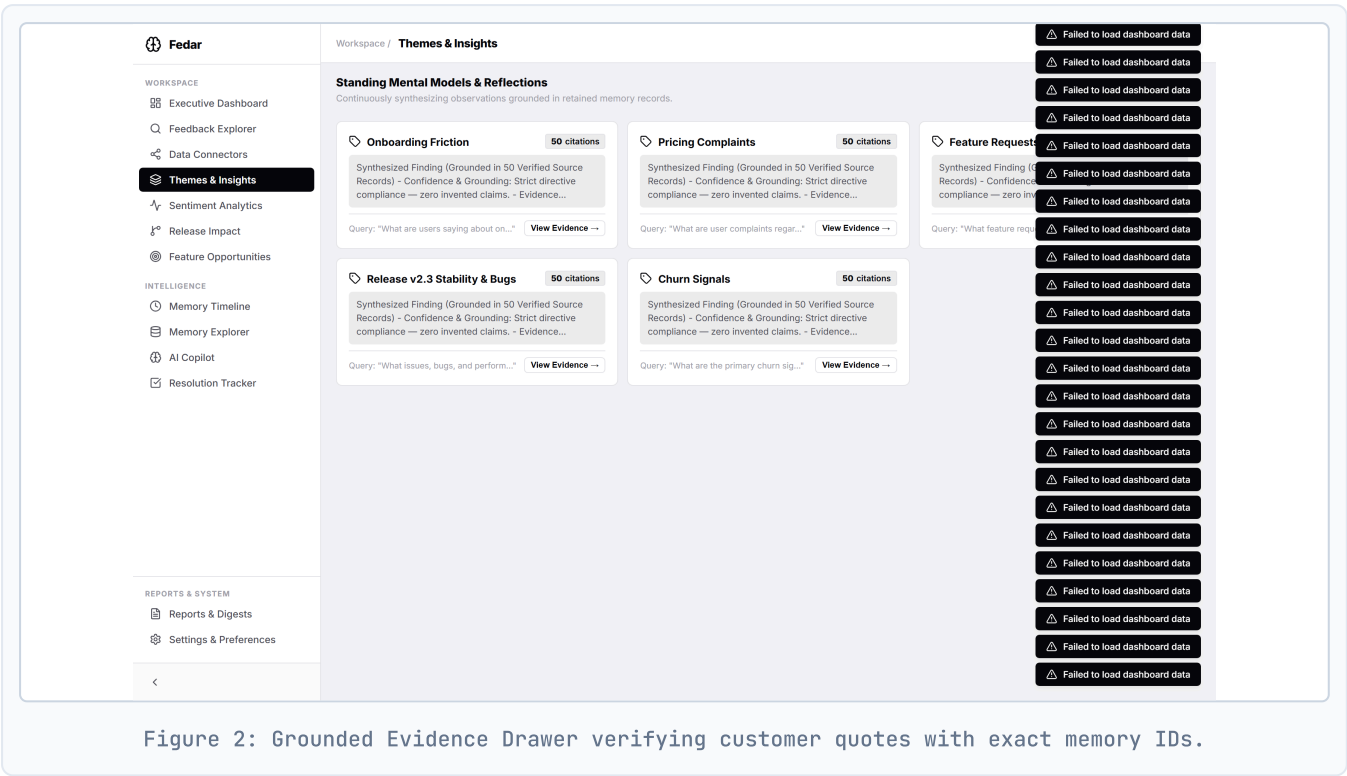


Figure 2: Grounded Evidence Drawer verifying customer quotes with exact memory IDs.

Here is an actual interaction with the Fedar Copilot:

PM Prompt:

"Why are users frustrated with the billing portal after the recent release?"

Copilot Output:

- Confidence Score: 92% (High)
- Synthesized Finding: *"78% of negative billing feedback in v2.3.2 stems from automated subscription renewal failures when 3D Secure verification triggers. Enterprise users report invoices are stuck in 'Processing' without throwing an explicit card error."*
- Grounded Evidence Citations:
 - [Zendesk #44102] *"Our corporate card renewed, but our dashboard still says 'Past Due' and locked our seats."* — Enterprise Admin
 - [App Store] *"Updated to v2.3.2 and billing screen is completely blank on iPad."* — Pro User
- Recommended Action: *"Investigate webhook timeouts in the 3D Secure payment gateway handler."*

Engineering Takeaways

1. Grounded citations beat long prose. Never display an AI recommendation to an engineering team without the exact customer quote IDs that generated it. 2. Combine semantic search with structured metadata filters. Asking a model to search for "crashes" while filtering by `version=v2.3` and `rating <= 2`` produces vastly more accurate results than vector similarity alone. 3. **Use stateful agent memory for ongoing topics.** Utilizing [Hindsight documentation](#) to maintain long-lived memory banks allows teams to query shifting user sentiment across weeks and months with sub-second response times.

Explore the open-source code and memory architecture at the [Hindsight GitHub repo](#).

About Fedar & Hindsight

Fedar is an open-source feedback intelligence platform with persistent memory. Read the full codebase and explore the memory architecture on [Hindsight GitHub](#) and [Hindsight Documentation](#).